

Rezolvare Completă și Detaliată

Concursul Național pentru Ocuparea Posturilor Didactice

Anul 2022 - Informatică și Tehnologia Informației

Cuprins

I. Rezolvare Titularizare Varianta 3 (13 iulie 2022)	2
SUBIECTUL I (30 de puncte)	2
1. Structura de date: Heap	2
2. Rețele de calculatoare: Concepte de bază	3
SUBIECTUL al II-lea (30 de puncte)	3
1. Programare: Rame concentrice	3
2. Algoritm eficient: Termeni recurență în interval	4
SUBIECTUL al III-lea (30 de puncte)	6
1. Itemi pentru secvența A: divizibilitate	6
2. Demers interdisciplinar pentru ergonomia postului de lucru	6

I. Rezolvare Titularizare Varianta 3 (13 iulie 2022)

SUBIECTUL I (30 de puncte)

1. Structura de date: Heap

- **Definire:** Un heap este un arbore binar aproape complet ce respectă proprietatea de heap: cheia fiecărui nod este mai mare sau egală cu cheile fiilor săi (Max-Heap) sau mai mică ori egală cu cheile acestora (Min-Heap).
- **Reprezentare:** Se utilizează un vector H în care rădăcina este $H[1]$. Pentru un nod de pe poziția i :
 - Fiu stânga: $2i$
 - Fiu dreapta: $2i + 1$
 - Părinte: $i \div 2$
- **Inserare (Max-Heap):** Se adaugă nodul pe prima poziție liberă la sfârșitul vectorului, apoi se reface proprietatea de heap prin urcarea elementului (up-heap / sift-up) prin comparații succesive cu părintele său.
- **Eliminare maxim (rădăcină):** Se înlocuiește rădăcina cu ultimul element din heap, se decrementează dimensiunea, iar apoi se coboară elementul (down-heap / sift-down / heapify) comparându-l cu cel mai mare dintre fiii săi până când structura este validă.
- **Problemă (HeapSort):**

▸ C++:

```
#include <iostream>
using namespace std;
void heapify(int A[], int n, int i) {
    int cel_mai_mare = i;
    int l = 2 * i + 1, r = 2 * i + 2;
    if (l < n && A[l] > A[cel_mai_mare]) cel_mai_mare = l;
    if (r < n && A[r] > A[cel_mai_mare]) cel_mai_mare = r;
    if (cel_mai_mare != i) {
        swap(A[i], A[cel_mai_mare]);
        heapify(A, n, cel_mai_mare);
    }
}
void heapSort(int A[], int n) {
    for (int i = n / 2 - 1; i >= 0; --i) heapify(A, n, i);
    for (int i = n - 1; i > 0; --i) {
        swap(A[0], A[i]);
        heapify(A, i, 0);
    }
}
int main() {
    int A[] = {12, 11, 13, 5, 6, 7};
    heapSort(A, 6);
    for (int x : A) cout << x << " ";
    return 0;
}
```

2. Rețele de calculatoare: Concepte de bază

- **Definiție:** Un grup de calculatoare conectate prin canale de comunicație pentru partajarea resurselor.
- **Avantaje:** Partajarea resurselor hardware (ex. imprimante) și software, stocare centralizată și comunicare rapidă (email, chat).
- **Tipuri:** LAN (Local Area Network), MAN (Metropolitan Area Network), WAN (Wide Area Network).
- **Protocoale:** TCP (controlul transmisiei), IP (adresa destinație).
- **Echipamente:** Switch, Router.

—

SUBIECTUL al II-lea (30 de puncte)

1. Programare: Rame concentrice

Soluție C++:

```
#include <iostream>
using namespace std;

int a[51][51];

void pqrama(int a[51][51], int p, int q, int x) {
    for (int j = p; j <= q; ++j) {
        a[p][j] = x;
        a[q][j] = x;
    }
    for (int i = p; i <= q; ++i) {
        a[i][p] = x;
        a[i][q] = x;
    }
}

int main() {
    int n;
    if (!(cin >> n)) return 0;

    int K = (n + 1) / 2;
    for (int i = 1; i <= K; ++i) {
        pqrama(a, i, n - i + 1, K - i + 1);
    }

    for (int i = 1; i <= n; ++i) {
        for (int j = 1; j <= n; ++j) {
            cout << a[i][j] << (j == n ? "" : " ");
        }
        cout << "\n";
    }
    return 0;
}
```

Soluție Pascal:

```
program RameConcentrice;
type
    TMatrice = array[1..50, 1..50] of integer;
```

```

var
  a: TMatrice;
  n, i, j, K: integer;

procedure pqrama(var a: TMatrice; p, q, x: integer);
var
  idx: integer;
begin
  for idx := p to q do
  begin
    a[p, idx] := x;
    a[q, idx] := x;
  end;
  for idx := p to q do
  begin
    a[idx, p] := x;
    a[idx, q] := x;
  end;
end;

begin
  read(n);
  K := (n + 1) div 2;
  for i := 1 to K do
    pqrama(a, i, n - i + 1, K - i + 1);

  for i := 1 to n do
  begin
    for j := 1 to n do
    begin
      write(a[i, j]);
      if j < n then write(' ');
    end;
    writeln;
  end;
end.

```

2. Algoritm eficient: Termeni recurență în interval

Metoda: Termenii au forma $a_n = n^2 + n + 1$, deci cresc strict odată cu n . Generăm succesiv termenii începând cu $n=0$, ne oprim când valoarea depășește limita superioară y și păstrăm numai termenii aflați în intervalul $[x, y]$. Deoarece cerința solicită afișarea în ordine descrescătoare, memorăm termenii găsiți și îi scriem invers.

Eficiență: Nu parcurgem toate numerele din interval, ci numai indicii n pentru care $n^2 + n + 1 \leq y$. Numărul acestor termeni este de ordin $O(\sqrt{y})$, iar generarea fiecăruia se face în timp constant. Memoria este $O(k)$, unde k este numărul termenilor afișați; se poate reduce la $O(1)$ dacă se determină mai întâi cel mai mare n valid și se coboară.

Soluție C++:

```

#include <iostream>
#include <fstream>
#include <vector>
using namespace std;

```

```
int main() {
    long long x, y;
    if (!(cin >> x >> y)) return 0;

    vector<long long> rez;
    long long n = 0;
    while (true) {
        long long val = n * n + n + 1;
        if (val > y) break;
        if (val >= x) rez.push_back(val);
        n++;
    }

    ofstream fout("titu2022.out");
    for (int i = rez.size() - 1; i >= 0; --i) {
        fout << rez[i] << (i == 0 ? "" : " ");
    }
    fout.close();

    return 0;
}
```

Soluție Pascal:

```
program RecurentaFisier;
var
    x, y, n, val: int64;
    rez: array[1..40000] of int64;
    count, i: integer;
    fout: text;
begin
    readln(x, y);
    n := 0;
    count := 0;
    while true do
        begin
            val := n * n + n + 1;
            if val > y then break;
            if val >= x then
                begin
                    count := count + 1;
                    rez[count] := val;
                end;
            n := n + 1;
        end;

    assign(fout, 'titu2022.out');
    rewrite(fout);
    for i := count downto 1 do
        begin
            write(fout, rez[i]);
            if i > 1 then write(fout, ' ');
        end;
    close(fout);
end.
```

SUBIECTUL al III-lea (30 de puncte)

1. Itemi pentru secvența A: divizibilitate

- **Obiectiv:** Un număr este divizibil cu 2 dacă ultima cifră este pară. (A/F) **Răspuns:** Adevărat.
- **Semiobiectiv:** Completați: a este divizibil cu b dacă $a \bmod b = [\text{spațiu liber}]$. **Răspuns:** 0.
- **Subiectiv:** Scrieți algoritmul care verifică dacă un număr este prim. **Răspuns:** testarea divizorilor până la radical, cu tratarea cazurilor $n < 2$.

2. Demers interdisciplinar pentru ergonomia postului de lucru

Particularități: integrează informatica, biologia și educația pentru sănătate; urmărește aplicarea cunoștințelor în comportamente concrete.

Metodă: proiectul. **Mijloc:** fișă de observație și calculator. **Formă:** grupe. **Activitate:** realizarea unui ghid de postură corectă la calculator.

Scenariu: Profesorul prezintă riscuri (oboseală oculară, dureri de spate), elevii analizează poziția monitorului/scaunului, propun reguli și realizează un afiș sau document informativ.